Your Bot Is a System

Capstone review: from language to procedures to a system – what ELIZA can do so far, and one function for many rules.

Marcus Birkenkrahe

COSC 1210 · CS I · Augustana University

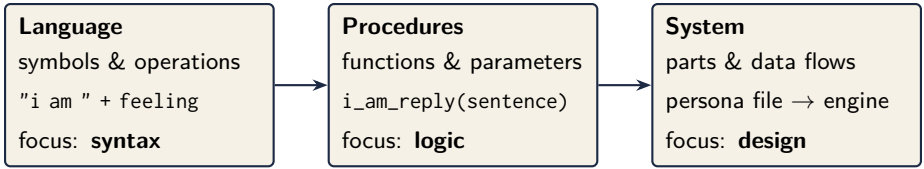
Fall 2026

Today

1. From language to system
2. Your bot is a system
3. What ELIZA can do so far
4. Three functions: what differs?
5. One function, many rules
6. From a call to a rule: what becomes data
7. Practice: a rule of your own
8. What the capstone adds

From language to system

In an AI future, AI writes more and more of the syntax and the procedures. Software engineering means: **you design the parts and their connections, and you write the tests that show it works.**

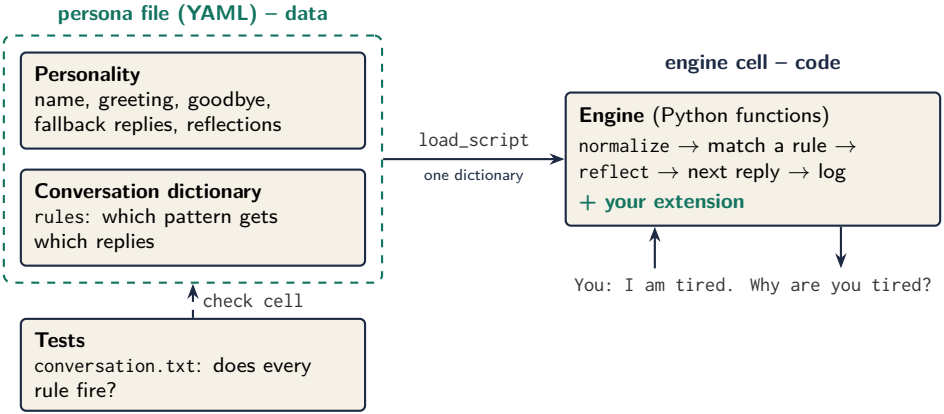


Each level is built from the one before: a system is made of procedures, and procedures are written in the language. At the system level, the bugs sit in the connections between the parts.

Software engineering: your bot is a system

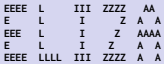
```
EEEE L    III ZZZZ  AA
E    L     I    Z  A A
EEE  L     I   Z   AAAA
E    L     I  Z    A  A
EEEE LLLL III ZZZZ A  A
```

A **system** is a set of parts, each with one job, and defined connections between them.



A new rule is **new data**, not new code: the engine stays the same.

What ELIZA can do so far



Date	Lecture	What ELIZA can do now
10/26-28	Spot a Keyword	notices keywords in your sentence: reads them from `keywords.txt`, finds them with `in` after `lower()`
10/30	The First Reply	picks the first keyword and answers with a fixed reply, or a fallback; doctests test the functions
11/04-06	Why Are You Tired?	reuses your words: "I am tired" → "Why are you tired?"; tidies up what you type, logs the conversation
11/09	Why Do You Need a Break?	one pattern per rule, `group(1)` cuts out your words; `\b` keeps "mother" out of "smothered"
next	**Can You Tell Me More?**	**rules = list items, tried top to bottom; replies take turns; words split into a list: "my" → "your"**

Colab: `birkenkrahe.github.io/go/elizareview`

Three functions: what differs?

```
def i_am_reply(sentence):
    match = re.search("^i am (.*)", normalize(sentence))
    if match is None:
        return None
    return f"Why are you {match.group(1)}?"

def feel_reply(sentence):
    match = re.search("^i feel (.*)", normalize(sentence))
    if match is None:
        return None
    return f"Do you often feel {match.group(1)}?"

def need_reply(sentence):
    match = re.search("i need (.*)", normalize(sentence))
    if match is None:
        return None
    return f"Why do you need {match.group(1)}?"
```

1. What differs between the three functions?

2. What is the same?

3. How many functions would ELIZA need for 50 rules?

Three functions: what differs? – answer

1. Only two strings differ:

Function	Pattern	Reply start
`i_am_reply`	`"^i am (.*)"`	`"Why are you "`
`feel_reply`	`"^i feel (.*)"`	`"Do you often feel "`
`need_reply`	`"i need (.*)"`	`"Why do you need "`

2. Everything else: `re.search`, `normalize`, the `None` check, `group(1)`, the `"?"` at the end.
3. 50 functions that are the same except for two strings.

→ What differs becomes a parameter: one function for all rules.

One function, many rules

→ **Switch to editor · code along**

The pattern and the reply start become parameters:

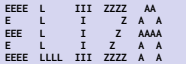
```
def rule_reply(pattern, before, sentence):
    match = re.search(pattern, normalize(sentence))
    if match is None:
        return None
    return before + match.group(1) + "?"

print(rule_reply("^i am (.*)", "Why are you ", "I'm awake."))
print(rule_reply("^i feel (.*)", "Do you often feel ", "I feel cold!"))
print(rule_reply("i need (.*)", "Why do you need ", "Well, I need coffee!"))
```

```
Why are you awake?
Do you often feel cold?
Why do you need coffee?
```

A rule is now two strings: a pattern and a reply start – ELIZA's real design.

From a call to a rule



In Python, all three are arguments:

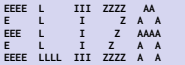
```
rule_reply("^i am (.*)", "Why are you ",
           "I'm awake.")
```

In your capstone's YAML file, it is a rule:

```
- pattern: '^i am (.*)'
  replies:
    - 'Why are you {x}?'
```

The pattern and the reply start are in the file. The sentence is not: it comes from the user while the bot runs.

What becomes data



rule_reply	Capstone	Where
pattern	rule["pattern"]	YAML file
before	a reply in rule["replies"]	YAML file
sentence	respond(sentence, script)	engine

→ What differs from rule to rule becomes data: a new rule needs no new code.

Practice: a rule of your own

→ **Switch to editor · 5 min, then 3 min discussion**

1. Call `rule_reply` so that “I can’t sleep.” gets this reply:

```
How do you know you can't sleep?
```

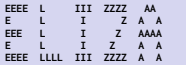
2. Now try to make “I feel cold.” get this reply:

```
Tell me more about feeling cold.
```

   Why can’t `rule_reply` do it?

3. Write the rule from 2. as a YAML rule for your capstone.

What the capstone adds



Task 1:

```
rule_reply("^i can't (.*)", "How do you know you can't ", "I can't sleep.")
```

Task 2: `rule_reply` always returns `before + group(1) + "?"`.

Task 3: In a YAML reply, `{x}` can go anywhere:

```
- pattern: '^i feel (.*)'
  replies:
    - 'Tell me more about feeling {x}.'
```

Also new: a rule has several replies, and `next_reply` takes them in turn.

→ Several replies, many rules: ELIZA needs a list – that is next.

Summary

- Your bot is a **system**: the persona file (YAML, data) and the engine (Python, code), connected by `load_script`; `conversation.txt` tests it
- ELIZA so far: keywords with `in`, a first reply with `return`, the user's words with slicing, then with `re.search` and `group(1)`
- Functions that differ only in values become one function with **parameters**: `rule_reply(pattern, before, sentence)`
- What differs from rule to rule becomes **data**: a pattern and a list of replies in the YAML file. A new rule needs no new code

Glossary

#	Term	Meaning
1	system	Parts with one job each, and their connections
2	persona file	The bot's data: name, replies, rules, in YAML
3	generalizing	Turning values that differ into parameters